

Incidents already happen in Slack. Now the response team does too.

SafeSlackForce is an AI incident response team inside Slack. A Commander and four specialist agents assign work to real people, chase anyone who does not answer, take hands-free emergency reports, and write the handoff, while a named human confirms every physical action.

5

coordinating agents

13

scoped tools

64

automated tests

0

physical actions an agent can confirm

The first ten minutes of an incident are a coordination failure

Someone posts "forklift tipped at Dock B, one person hurt". Everyone sees it. What happens next depends on who happens to be paying attention.

Nobody owns it

Messages get reactions, not owners. The failure mode is a report everyone saw and no one accepted.

Facts drift

Location, injuries and whether help was called change across replies and edits. Contradictions go unnoticed.

The record is rebuilt later

The handoff report is written from memory after the fact, without sources, and open work gets lost.

Report in the thread. The team forms around it.

01

Report

@mention the bot in the incident channel

02

Coordinate

Commander records sourced facts and delegates

03

Assign

Named owners get tasks with Slack buttons

04

Escalate

No acknowledgement, the backup is chased

05

Correct

Edits and contradictions flag only affected work

06

Hand off

Sourced report; a supervisor accepts in Slack

Live on screen: the 3D command center shows every agent's state as the thread changes.

This cannot be done from a chatbox

A chatbot can give advice to one person. Coordination needs other people, and those people are already in Slack.

Assign to a colleague

Tasks go to real Slack users with Acknowledge, Request review and Confirm action buttons.

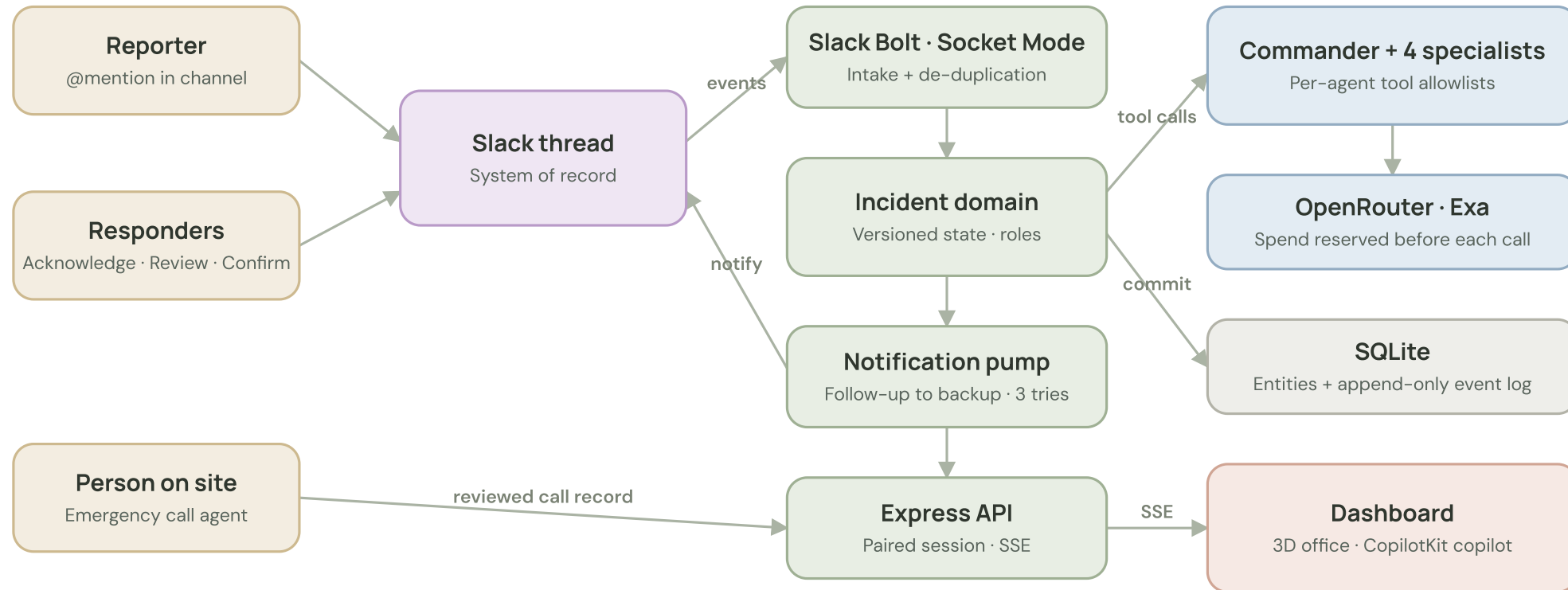
Wait, then escalate

Sent and acknowledged are different states. Silence triggers one follow-up to the backup owner.

One shared record

Who was assigned, who accepted and who confirmed is the thread history everyone already sees.

One thread, one process, five agents



■ People
 ■ Slack
 ■ API process
 ■ Agents and providers
 ■ Storage
 ■ Browser

Socket Mode needs no public webhook · every change is versioned and logged · the dashboard streams events and resumes after a disconnect. Full register: 18 design decisions on the System design page.

The model proposes. The server decides.

Commander

Reads the report, records location and facts, delegates, asks humans for missing facts

read_incident
read_procedure
read_office
update_location
record_fact delegate
ask_human

Procedure

Applies the approved procedure only when the report fits its scope; creates human-owned tasks

read_incident
read_procedure
read_office
apply_procedure
flag_contradiction
ask_human

Evidence

Compares messages, links observations to facts, flags real contradictions, describes photos as unverified

read_incident
record_fact
flag_contradiction
ask_human inspect_image

Communications

Notifies the configured roster about open tasks, without duplicates

read_incident
read_procedure
read_office notify

Records

Reads the whole timeline and saves a sourced Markdown handoff report

read_incident
read_history read_office
save_report

IDs, timers, roster checks and retries are server code. Slack text is treated as evidence, never as instructions.

Agents move digital work. People own the physical world.

Only people confirm

Completing a physical task needs the owner or a supervisor and a written note. Agents cannot close an incident.

Reviewed before relay

Voice updates, emergency call records and copilot questions are shown first and need an explicit OK.

Everything is sourced

Facts are recorded as reported, with the Slack message they came from. Reports cite their sources.

An emergency line and a copilot, same guardrails

Emergency call agent

Hands-free for the person on site: 4 spoken triage questions, answers by voice or text. Words like "unconscious" or "trapped" trigger "Call 911 now" from a fixed rule that works even if the model is down. The reviewed call record goes into the thread for the Commander.

CopilotKit incident copilot

Reads live incident state, focuses agent desks and opens panels. Before it asks an agent anything that posts to Slack, it shows an Approve and send card. Runs inside our API behind the same session and spend cap.

Designed for timeouts, restarts and double clicks

Uncertain, not resent

A timed-out Slack send is flagged for a supervisor instead of paging someone twice. At most 3 attempts.

Stale is rejected

Idempotency keys and version checks stop old buttons and repeated submissions from overwriting newer state.

Honest recovery

After a restart, interrupted work is marked failed or uncertain. A failed tool can never become a successful agent.

44

backend tests

20

frontend tests

\$

reserved before every model and search call

What it is today, and what comes next

Today

- One Slack workspace and incident channel
- Synthetic procedure, roster and office directory
- Guides people to call 911; does not dispatch services
- Dashboard deploys to Vercel; API runs as a persistent process

Next

- Real phone escalation when a critical task goes unacknowledged
- Microsoft Teams through CopilotKit channels
- A library of approved procedures per site and incident type
- Post-incident review generated from the event log

THANK YOU

**A real report becomes coordinated work,
a human-accepted handoff and a sourced record.**

Open the workspace

System design

github.com/ritzzi23/SafeSlackForce

Built by Om and Ritesh during the hackathon. OpenRouter · Exa · CopilotKit · Slack Bolt · React · three.js